Ebbinghaus Benchmark — Python Reference Manual

Function and Module Documentation

March 2026

Contents

Configuration **3**
- config.defaults . . . 3
  - Constants . . . 3

Phase 1: Stimulus Creation **3**
- src.generate_design . . . 3
  - generate_design . . . 3
- src.generate_trial . . . 4
  - pick_contrasting_color . . . 4
  - pick_fill . . . 4
  - min_distance_no_surround_overlap . . . 4
  - group_separation . . . 4
  - generate_trial . . . 4
- src.render_stimuli . . . 5
  - render_stimuli . . . 5
- src.draw_trial . . . 6
  - compute_group_positions . . . 6
  - compute_surround_positions . . . 6
  - compute_all_extents . . . 6
  - draw_trial . . . 6
- src.draw_shape . . . 7
  - draw_shape . . . 7
- src.verify_trial . . . 7
  - verify_trial . . . 7
- src.classify_tier . . . 8
  - classify_tier . . . 8

Phase 2: Evaluation **8**
- src.evaluate . . . 8
  - fill_prompt . . . 8
  - parse_response . . . 8
  - build_dataset . . . 9
  - ebbinghaus_solver . . . 9

ebbinghaus_scorer 9
run_evals 9

Phase 3: Analysis **10**
src.analyze 10
load_eval_data 10
compute_dprime 10
analyze_results 10

Utilities **11**
src.strip_answer 11
strip_answer_from_path 11
strip_answer_from_images 11

Configuration

config.defaults

Source: py/config/defaults.py

Ebbinghaus Benchmark — Default Configuration

Constants

- SHAPE_POOL = ['circle', 'square']
- TEST_SIZE_RANGE = (0.03, 0.08)
- SURROUND_SIZE_RANGE = (0.02, 0.15)
- SURROUND_N_RANGE = (4, 8)
- SURROUND_DISTANCE_RANGE = (0.08, 0.22)
- COLOR_POOL = ['#000000', '#4D4D4D', '#999999', '#4682B4', '#B22222']
- FILL_POOL = ['#000000', '#4D4D4D', '#999999', '#4682B4', '#B22222', None]
- BACKGROUND_POOL = ['#FFFFFF', '#000000']
- CANVAS_SIZES = [512, 768, 1024]
- ORIENTATION_POOL = ['horizontal', 'vertical', 'diagonal']
- FILE_FORMAT_POOL = ['png', 'svg', 'webp']
- FLOAT_TOLERANCE = 1e-09
- DEFAULT_N_PER_TIER = 50
- DEFAULT_FILE_FORMAT = 'png'
- DEFAULT_CANVAS_WIDTH = 768
- DEFAULT_CANVAS_HEIGHT = 768

Phase 1: Stimulus Creation

src.generate_design

Source: py/src/generate_design.py

generate_design.py — Design matrix builder

generate_design

```
generate_design(
    seed: int | None = None,
    n_per_tier: int = DEFAULT_N_PER_TIER,
    file_format: str = DEFAULT_FILE_FORMAT,
    image_dir: str = 'images',
    **kwargs
)
```

Generate a complete design matrix of trials.

Creates a balanced trial table with representation across tiers. Each trial gets a unique trial_id and file_path.

Args:

- `seed`: Master seed for the entire design.
- `n_per_tier`: Number of trials per tier (0, 1, 2, 3).
- `file_format`: Default file format for images.
- `image_dir`: Directory for images (relative to project root). **kwargs: Additional arguments passed to generate_trial().

Returns:

A DataFrame with complete trials schema, verified and classified.

---

src.generate_trial

Source: `py/src/generate_trial.py`

generate_trial.py — Constrained random trial generator

pick_contrasting_color

```
pick_contrasting_color(rng, pool: list[str], bg: str)
```

Pick a color from a pool, ensuring contrast against background.

pick_fill

```
pick_fill(rng, pool: list)
```

Pick a fill color (may be None for transparent).

min_distance_no_surround_overlap

```
min_distance_no_surround_overlap(
    n: int,
    s: float,
    shape: str = 'circle',
    gap_frac: float = 0.1
)
```

Minimum distance so adjacent surrounds in a ring don't overlap.

group_separation

```
group_separation(orientation: str, canvas_w: int, canvas_h: int)
```

Compute distance between group centers A and B.

generate_trial

```
generate_trial(
    seed: int | None = None,
    tier: int | None = None,
```

```
    orientation: str | None = None,
    shape_pool: list[str] = SHAPE_POOL,
    size_range: tuple = TEST_SIZE_RANGE,
    surround_size_range: tuple = SURROUND_SIZE_RANGE,
    surround_n_range: tuple = SURROUND_N_RANGE,
    surround_distance_range: tuple = SURROUND_DISTANCE_RANGE,
    canvas_sizes: list[int] = CANVAS_SIZES,
    color_pool: list[str] = COLOR_POOL,
    fill_pool: list = FILL_POOL,
    background_pool: list[str] = BACKGROUND_POOL,
    orientation_pool: list[str] = ORIENTATION_POOL,
    file_format: str = DEFAULT_FILE_FORMAT
)
```

Generate a single random trial.

Args:

- `seed`: Integer seed for reproducibility. None = generate a random seed.
- `tier`: Target tier (0, 1, 2, 3). None = unconstrained.
- `orientation`: Fixed orientation, or None for random.
- `shape_pool`: Shapes to sample from.
- `size_range`: (min, max) test size proportions.
- `surround_size_range`: (min, max) surround size proportions.
- `surround_n_range`: (min, max) surround count.
- `surround_distance_range`: (min, max) extra distance padding proportions.
- `canvas_sizes`: Candidate canvas dimensions.
- `color_pool`: Border colors.
- `fill_pool`: Fill colors (may include None).
- `background_pool`: Background colors.
- `orientation_pool`: Orientations to sample from.
- `file_format`: File format for rendering.

Returns:

A one-row DataFrame with the complete trials schema.

---

src.render_stimuli

Source: `py/src/render_stimuli.py`

render_stimuli.py — Batch renderer

render_stimuli

```
render_stimuli(trials: pd.DataFrame, verbose: bool = True)
```

Render stimulus images for a trial table.

Args:

- `trials`: DataFrame with the trials table schema (or a subset).
- `verbose`: Print progress messages.

Returns:

The input trials DataFrame (unchanged). Side effect: writes image files to paths specified in each row's `file_path` column.

---

src.draw_trial

Source: `py/src/draw_trial.py`

draw_trial.py — Compose a full Ebbinghaus stimulus from a trial row

compute_group_positions

```
compute_group_positions(
    orientation: str,
    canvas_width: int,
    canvas_height: int
)
```

Compute center positions for groups A and B based on orientation.

compute_surround_positions

```
compute_surround_positions(
    cx: float,
    cy: float,
    n: int,
    distance: float
)
```

Compute positions for N surround shapes equally spaced around a center.

compute_all_extents

```
compute_all_extents(
    trial: dict,
    positions: dict,
    padding: float = 0.05
)
```

Compute bounding box that contains all shapes in a trial.

draw_trial

```
draw_trial(trial)
```

Draw a complete Ebbinghaus stimulus from trial parameters.

Args:

- `trial`: A one-row DataFrame or dict with the trials table schema.

Returns:

A matplotlib Figure.

---

src.draw_shape

Source: `py/src/draw_shape.py`

draw_shape.py — Atomic shape-drawing function

draw_shape

```
draw_shape(
    ax,
    shape: str,
    x: float,
    y: float,
    size: float,
    color: str = 'black',
    fill = None,
    alpha: float = 1.0
)
```

Draw a single shape on a matplotlib Axes.

Args:

- `ax`: matplotlib Axes to draw on.
- `shape`: "circle" or "square".
- `x`: Center x-coordinate.
- `y`: Center y-coordinate.
- `size`: Radius (circle) or half side-length (square), in plot units.
- `color`: Border/stroke color.
- `fill`: Fill color. None = no fill (transparent).
- `alpha`: Opacity (0-1).

---

src.verify_trial

Source: `py/src/verify_trial.py`

verify_trial.py — Deterministic ground-truth verifier

verify_trial

```
verify_trial(trials: pd.DataFrame, tol: float = FLOAT_TOLERANCE)
```

Compute ground truth for trial(s).

Args:

- `trials`: DataFrame with at least `test_a_size` and `test_b_size`.
- `tol`: Floating-point tolerance for equality.

Returns:

The input DataFrame with `true_larger` and `true_diff_ratio` columns added (or overwritten).

---

src.classify_tier

Source: `py/src/classify_tier.py`

classify_tier.py — Deterministic difficulty tier classifier

classify_tier

```
classify_tier(trials: pd.DataFrame, tol: float = FLOAT_TOLERANCE)
```

Classify trials into difficulty tiers.

Args:

- `trials`: DataFrame with trial parameters. Must include `true_larger`, `surround_a_n`, `surround_b_n`, `surround_a_size`, `surround_b_size`.
- `tol`: Floating-point tolerance for surround size equality.

Returns:

The input DataFrame with `tier` column added (or overwritten).

Phase 2: Evaluation

src.evaluate

Source: `py/src/evaluate.py`

evaluate.py — Phase 2: Evaluation Pipeline (Inspect AI-based)

fill_prompt

```
fill_prompt(template: str, trial: dict)
```

Fill prompt template placeholders for a specific trial.

Replaces {direction_a}, {direction_b}, {test_a_shape}, {test_b_shape} with values from the trial.

parse_response

```
parse_response(
    raw_response: str,
    orientation: str,
```

```
    response_format: str = 'forced_choice'
)
```

Extract a structured response_larger value from raw model text.

Maps directional words to "a" or "b" based on the trial's orientation, then normalises to: "a", "b", "equal", "unknown", "parse_error".

build_dataset

```
build_dataset(
    trials: pd.DataFrame,
    prompt: pd.Series,
    image_dir: str = 'images_eval'
)
```

Build Inspect AI samples from trials and a single prompt variant.

Each Sample represents one (trial, prompt) pair.

ebbinghaus_solver

```
ebbinghaus_solver()
```

Solver that passes through the image + text to the model.

ebbinghaus_scorer

```
ebbinghaus_scorer()
```

Score Ebbinghaus evaluation results using deterministic parsing.

run_evals

```
run_evals(
    trials: pd.DataFrame,
    prompts: pd.DataFrame,
    models: list[dict],
    image_dir: str = 'images_eval',
    epochs: int = 1
)
```

Run the full evaluation across prompts × models.

Creates one Inspect AI Task per (prompt, model) combination, evaluates it, and returns a list of eval logs.

Args:

- `trials`: DataFrame from trials.csv.
- `prompts`: DataFrame from prompts.csv.
- `models`: List of dicts with 'provider', 'model', optionally 'temperature', 'max_tokens'.
- `image_dir`: Directory containing answer-stripped images.
- `epochs`: Number of times to repeat each sample.

Returns:

A list of Inspect AI eval log objects.

Phase 3: Analysis

src.analyze

Source: *`py/src/analyze.py`*

analyze.py — Phase 3: Analysis

load_eval_data

```
load_eval_data(
    logs = None,
    trials_path: str = 'data/trials.csv',
    prompts_path: str = 'data/prompts.csv',
    evals_path: str | None = None
)
```

Load and join data from Inspect AI logs or legacy CSV files.

Args:

- `logs`: List of Inspect AI eval log objects (from run_evals()).
- `trials_path`: Path to trials.csv.
- `prompts_path`: Path to prompts.csv.
- `evals_path`: Path to legacy evals.csv (only if logs is None).

Returns:

A DataFrame with all columns needed for analysis.

compute_dprime

```
compute_dprime(data: pd.DataFrame)
```

Compute d-prime (signal detection sensitivity) per model.

Uses Hautus (1995) log-linear correction.

analyze_results

```
analyze_results(
    logs = None,
    trials_path: str = 'data/trials.csv',
    evals_path: str | None = None,
    prompts_path: str = 'data/prompts.csv',
    output_dir: str = 'output'
)
```

Analyze evaluation results.

Computes accuracy and bias metrics and generates key plots. All outputs (plots as PNG, tables as CSV) are saved to output_dir.

Args:

- `logs`: List of Inspect AI eval log objects (from run_evals()).
- `trials_path`: Path to trials.csv.
- `evals_path`: Path to legacy evals.csv (only if logs is None).
- `prompts_path`: Path to prompts.csv.
- `output_dir`: Directory for plots and summary tables.

Returns:

Dict with: data, metrics, plots.

Utilities

src.strip_answer

Source: `py/src/strip_answer.py`

strip_answer.py — Strip ground-truth answer from image filenames

strip_answer_from_path

```
strip_answer_from_path(path: str)
```

Strip the answer and tier from a single file path.

Converts "images/42_equal_t1.png" -> "images/42.png" Also handles the old format without tier.

Args:

- `path`: File path in the format ___t.

Returns:

Cleaned path in the format .

strip_answer_from_images

```
strip_answer_from_images(
    trials: pd.DataFrame,
    output_dir: str = 'images_eval',
    verbose: bool = True
)
```

Create answer-stripped copies of image files for AI evaluation.

Copies each image to a clean filename (without the ground-truth answer) in a target directory.

Args:

- `trials`: DataFrame with trials schema (needs `file_path` column).
- `output_dir`: Directory for the stripped copies.
- `verbose`: Print progress.

Returns:

The input DataFrame with `file_path` updated to the stripped copies. Original path preserved in `file_path_original`.